

Lecture 9: Policy Gradients and Actor Critics

UCL COMP0089 — Reinforcement Learning

Lecturer: Hado van Hasselt (2021)

Lecture Notes

Background reading. Sutton & Barto (2018), Chapter 13.

Contents

1	Motivation: Why Learn a Policy Directly?	2
1.1	General Overview	2
1.2	Terminology: Value-Based, Policy-Based, Actor-Critic	2
1.3	Advantages and Disadvantages of Policy-Based RL	2
2	Benefits of Stochastic Policies	3
2.1	Why Stochastic Policies?	3
2.2	Example: Aliased Grid World	3
3	Policy Objective Functions	3
3.1	Episodic Setting	3
3.2	Continuing (Average-Reward) Setting	4
4	Policy Gradients	4
4.1	Gradient Ascent on the Objective	4
4.2	The Score Function Trick (REINFORCE)	4
4.3	Variance Reduction via Baselines	5
4.4	Example: Softmax Policy	5
5	Policy Gradient Theorem	5
5.1	Episodic Policy Gradient Theorem	6
5.2	Practical Episodic Policy Gradient	7
5.3	Average-Reward Policy Gradient Theorem	7
6	Actor-Critic Methods	7
6.1	Reducing Variance with a Critic	7
6.2	The Critic	8
6.3	One-Step Actor-Critic Algorithm	8
6.4	Trust Region Methods	8
7	Continuous Action Spaces	8
7.1	Gaussian Policy	9
7.2	Deterministic Policy Gradient (DPG)	9
7.3	Continuous Actor-Critic Learning Automaton (CACLA)	9
8	Summary	10

1 Motivation: Why Learn a Policy Directly?

Previous lectures approximated parametric value functions (cf. `rl_lecture_7`, Definition `def:vfa`):

$$v_{\mathbf{w}}(s) \approx v_{\pi}(s), \quad q_{\mathbf{w}}(s, a) \approx q_{\pi}(s, a), \quad (1)$$

and derived a policy implicitly (e.g. ϵ -greedy on $q_{\mathbf{w}}$). In this lecture we *directly* parametrise the policy

$$\pi_{\boldsymbol{\theta}}(a \mid s) = p(a \mid s, \boldsymbol{\theta}), \quad (2)$$

and optimise the parameters $\boldsymbol{\theta}$ to maximise the expected return. We focus throughout on model-free reinforcement learning (cf. `rl_lecture_8`, Definition `def:model-free-rl`).

1.1 General Overview

There are three broad classes of RL algorithms:

- (i) **Model-based RL.** *Pros:* Learning a model is supervised learning; the model captures all information from data. *Cons:* Wastes capacity on irrelevant details; planning (computing a policy from the model) is expensive.
- (ii) **Value-based RL.** *Pros:* Easy to derive a policy (e.g. $\pi(a \mid s) = \mathbb{I}(a = \arg \max_a q(s, a))$); close to the true objective; well-understood. *Cons:* Still not the true objective—small value error can cause large policy error; may devote capacity to irrelevant details.
- (iii) **Policy-based RL.** *Pros:* Directly optimises the right objective. *Cons:* see Section 1.3 below.

These approaches generalise in complementary ways. Sometimes learning a model is easiest (e.g. simple dynamics); sometimes learning a policy is easiest (e.g. if the optimal policy is simply “always move forward”).

1.2 Terminology: Value-Based, Policy-Based, Actor-Critic

Definition 1.1 (Value-Based RL). In **value-based** methods the agent learns a value function and derives the policy implicitly (e.g. ϵ -greedy). No explicit policy parameters are maintained.

Definition 1.2 (Policy-Based RL). In (pure) **policy-based** methods the agent maintains an explicit parametric policy $\pi_{\boldsymbol{\theta}}$ and learns $\boldsymbol{\theta}$ directly, *without* maintaining a value function.

Definition 1.3 (Actor-Critic). An **actor-critic** method learns *both* a policy (the *actor*, $\pi_{\boldsymbol{\theta}}$) and a value function (the *critic*, $v_{\mathbf{w}}$ or $q_{\mathbf{w}}$). The critic assists the actor by providing lower-variance estimates of the policy gradient.

Remark 1.1 (Venn Diagram). Actor-critic lives at the intersection of value-based and policy-based methods: it maintains both a value function and a policy.

1.3 Advantages and Disadvantages of Policy-Based RL

Advantages.

- (a) Directly optimises the true objective.
- (b) Naturally extends to high-dimensional or continuous action spaces (cf. Section 7).
- (c) Can represent stochastic policies (cf. Section 2.1).
- (d) Sometimes the optimal policy is much simpler than the value function or model.

Disadvantages.

- (a) Can get stuck in local optima.
- (b) Learned knowledge can be specific and may not generalise well.
- (c) In isolation, does not necessarily extract all useful information from data.

2 Benefits of Stochastic Policies

2.1 Why Stochastic Policies?

Remark 2.1 (When Stochastic Policies Are Needed). Four reasons motivate stochastic policies:

- (i) In fully observable MDPs an optimal deterministic policy always exists. However, most practical problems are **not fully observable**—especially when function approximation is used. The optimal policy under partial observability may be stochastic.
- (ii) The search space over stochastic policies is **smoother**, enabling gradient-based optimisation.
- (iii) Stochastic policies provide natural **exploration** during learning.

2.2 Example: Aliased Grid World

Example 2.1 (Aliased Grid World). Consider a grid world with two visually identical (“aliased”) grey states. The agent’s feature vector encodes only which walls are present:

$$\phi(s) = \left(\underbrace{1}_{\text{wall up}}, \underbrace{0}_{\text{wall right}}, \underbrace{1}_{\text{wall down}}, \underbrace{0}_{\text{wall left}} \right).$$

Under this aliasing, any *deterministic* policy must choose the same direction in both grey states (e.g. always left, or always right). Either choice can cause the agent to get stuck, never reaching the goal.

An optimal *stochastic* policy assigns

$$\pi_{\theta}(\text{right} \mid \text{wall up \& down}) = 0.5, \quad \pi_{\theta}(\text{left} \mid \text{wall up \& down}) = 0.5,$$

and reaches the goal state in a few steps with high probability.

This differs from random tie-breaking in value-based methods: policy gradient methods can learn *any* optimal stochastic mixture, not only the uniform one.

3 Policy Objective Functions

Given a parametric policy $\pi_{\theta}(a \mid s)$, we seek $\theta^* = \arg \max_{\theta} J(\theta)$. How we define J depends on the problem setting.

3.1 Episodic Setting

Definition 3.1 (Episodic-Return Objective). Let d_0 denote the start-state distribution. The **episodic-return objective** is

$$J_G(\theta) = \mathbb{E}_{S_0 \sim d_0, \pi_{\theta}} \left[\sum_{t=0}^{\infty} \gamma^t R_{t+1} \right] = \mathbb{E}_{S_0 \sim d_0} [v_{\pi_{\theta}}(S_0)], \quad (3)$$

i.e. the expected value of the start state under the policy.

3.2 Continuing (Average-Reward) Setting

Definition 3.2 (Average-Reward Objective). The **average-reward objective** is

$$J_R(\boldsymbol{\theta}) = \mathbb{E}_{\pi_{\boldsymbol{\theta}}}[R_{t+1}] = \sum_s d_{\pi_{\boldsymbol{\theta}}}(s) \sum_a \pi_{\boldsymbol{\theta}}(a | s) \sum_r p(r | s, a) r, \quad (4)$$

where $d_{\pi}(s) = p(S_t = s | \pi)$ is the **stationary (steady-state) distribution** under π , representing the long-run fraction of time spent in state s .

4 Policy Gradients

4.1 Gradient Ascent on the Objective

Policy-based RL is an optimisation problem: find $\boldsymbol{\theta}$ that maximises $J(\boldsymbol{\theta})$. We use **stochastic gradient ascent**:

$$\Delta \boldsymbol{\theta} = \alpha \nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta}), \quad (5)$$

where $\alpha > 0$ is a step-size and

$$\nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta}) = \begin{pmatrix} \frac{\partial J(\boldsymbol{\theta})}{\partial \theta_1} \\ \vdots \\ \frac{\partial J(\boldsymbol{\theta})}{\partial \theta_n} \end{pmatrix}. \quad (6)$$

Stochastic policies help ensure $J(\boldsymbol{\theta})$ is a smooth function of $\boldsymbol{\theta}$, which is important for gradient-based methods.

Remark 4.1 (Gradient-Free Alternatives). Some approaches do not use gradients at all: hill climbing, simulated annealing, genetic algorithms, and evolutionary strategies. These are generally less sample-efficient than gradient methods for policy search.

4.2 The Score Function Trick (REINFORCE)

Consider first the *contextual bandit* setting (one-step MDP) where $J(\boldsymbol{\theta}) = \mathbb{E}_{\pi_{\boldsymbol{\theta}}}[R(S, A)]$ and the state distribution d does not depend on π .

We cannot simply sample R_{t+1} and differentiate, because a sampled reward is a number that does not depend on $\boldsymbol{\theta}$. Instead, we use the following identity.

Lemma 4.1 (Score Function Trick / Log-Likelihood Trick). *Let d be a state distribution independent of $\boldsymbol{\theta}$, and let $r_{sa} = \mathbb{E}[R(S, A) | S = s, A = a]$. Then*

$$\nabla_{\boldsymbol{\theta}} \mathbb{E}_{\pi_{\boldsymbol{\theta}}}[R(S, A)] = \mathbb{E}_{\pi_{\boldsymbol{\theta}}}[R(S, A) \nabla_{\boldsymbol{\theta}} \log \pi_{\boldsymbol{\theta}}(A | S)]. \quad (7)$$

Proof.

$$\begin{aligned} \nabla_{\boldsymbol{\theta}} \mathbb{E}_{\pi_{\boldsymbol{\theta}}}[R(S, A)] &= \nabla_{\boldsymbol{\theta}} \sum_s d(s) \sum_a \pi_{\boldsymbol{\theta}}(a | s) r_{sa} \\ &= \sum_s d(s) \sum_a r_{sa} \nabla_{\boldsymbol{\theta}} \pi_{\boldsymbol{\theta}}(a | s) \\ &= \sum_s d(s) \sum_a r_{sa} \pi_{\boldsymbol{\theta}}(a | s) \frac{\nabla_{\boldsymbol{\theta}} \pi_{\boldsymbol{\theta}}(a | s)}{\pi_{\boldsymbol{\theta}}(a | s)} \\ &= \sum_s d(s) \sum_a \pi_{\boldsymbol{\theta}}(a | s) r_{sa} \nabla_{\boldsymbol{\theta}} \log \pi_{\boldsymbol{\theta}}(a | s) \\ &= \mathbb{E}_{d, \pi_{\boldsymbol{\theta}}}[R(S, A) \nabla_{\boldsymbol{\theta}} \log \pi_{\boldsymbol{\theta}}(A | S)]. \end{aligned} \quad (8)$$

The key step is multiplying and dividing by $\pi_{\boldsymbol{\theta}}(a | s)$ and using $\nabla_{\boldsymbol{\theta}} \log f = \frac{\nabla_{\boldsymbol{\theta}} f}{f}$. $\square$

Since the right-hand side of (7) is an expectation, we can estimate it from samples. The stochastic gradient update is

$$\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t + \alpha R_{t+1} \nabla_{\boldsymbol{\theta}} \log \pi_{\boldsymbol{\theta}_t}(A_t | S_t). \quad (9)$$

In expectation this follows the true gradient, making it an unbiased stochastic gradient ascent algorithm. This is known as **REINFORCE** (Williams, 1992).

Intuition. The update increases the probability of actions that yield high rewards and decreases the probability of actions that yield low rewards.

4.3 Variance Reduction via Baselines

Proposition 4.2 (Baseline Does Not Change Expected Gradient). *For any function $b(S_t)$ that does not depend on the action A_t ,*

$$\mathbb{E}_{\pi}[b(S_t) \nabla_{\boldsymbol{\theta}} \log \pi_{\boldsymbol{\theta}}(A_t | S_t)] = 0. \quad (10)$$

Proof.

$$\mathbb{E}[b \nabla_{\boldsymbol{\theta}} \log \pi(A_t | S_t)] = \mathbb{E}\left[\sum_a \pi(a | S_t) b \nabla_{\boldsymbol{\theta}} \log \pi(a | S_t)\right] = \mathbb{E}\left[b \nabla_{\boldsymbol{\theta}} \sum_a \pi(a | S_t)\right] = \mathbb{E}[b \nabla_{\boldsymbol{\theta}} 1] = 0.$$

□

Corollary 4.3 (REINFORCE with Baseline). *By Proposition 4.2, we can subtract any state-dependent baseline $b(S_t)$ from the reward without introducing bias:*

$$\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t + \alpha (R_{t+1} - b(S_t)) \nabla_{\boldsymbol{\theta}} \log \pi_{\boldsymbol{\theta}_t}(A_t | S_t). \quad (11)$$

A good baseline (e.g. $b(s) \approx v_{\pi}(s)$) significantly reduces the variance of the gradient estimator.

4.4 Example: Softmax Policy

Example 4.1 (Softmax Policy). Define action preferences $h(s, a)$ and let

$$\pi_{\boldsymbol{\theta}}(a | s) = \frac{e^{h(s, a)}}{\sum_b e^{h(s, b)}}. \quad (12)$$

The score function (gradient of the log-policy) is then

$$\nabla_{\boldsymbol{\theta}} \log \pi_{\boldsymbol{\theta}}(A_t | S_t) = \underbrace{\nabla_{\boldsymbol{\theta}} h(S_t, A_t)}_{\text{gradient of preference}} - \underbrace{\sum_a \pi_{\boldsymbol{\theta}}(a | S_t) \nabla_{\boldsymbol{\theta}} h(S_t, a)}_{\text{expected gradient of preference}}. \quad (13)$$

5 Policy Gradient Theorem

The score-function approach from Section 4.2 generalises to multi-step MDPs. In the MDP setting, the state distribution depends on the policy, yet—remarkably—the gradient can still be expressed without differentiating through the dynamics.

There are two versions of the theorem (Sutton et al., 2000): one for the episodic-return objective and one for the average-reward objective.

5.1 Episodic Policy Gradient Theorem

Theorem 5.1 (Policy Gradient Theorem — Episodic). *For any differentiable policy $\pi_{\theta}(a | s)$, let d_0 be the starting distribution over states. Then the gradient of $J(\theta) = \mathbb{E}[G_0 | S_0 \sim d_0]$ (cf. Definition 3.1) is*

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{\pi_{\theta}} \left[\sum_{t=0}^T \gamma^t q_{\pi_{\theta}}(S_t, A_t) \nabla_{\theta} \log \pi_{\theta}(A_t | S_t) \mid S_0 \sim d_0 \right], \quad (14)$$

where

$$q_{\pi}(s, a) = \mathbb{E}_{\pi}[G_t | S_t = s, A_t = a] = \mathbb{E}_{\pi}[R_{t+1} + \gamma q_{\pi}(S_{t+1}, A_{t+1}) | S_t = s, A_t = a]. \quad (15)$$

Proof of Theorem 5.1. Consider a trajectory $\tau = (S_0, A_0, R_1, S_1, A_1, R_2, \dots)$ with return $G(\tau) = \sum_{t=0}^T \gamma^t R_{t+1}$. By the score function trick (Lemma 4.1) applied to the trajectory distribution:

$$\nabla_{\theta} J(\theta) = \nabla_{\theta} \mathbb{E}[G(\tau)] = \mathbb{E}[G(\tau) \nabla_{\theta} \log p(\tau)]. \quad (16)$$

The log-probability of a trajectory factorises as

$$\log p(\tau) = \log p(S_0) + \sum_{t=0}^T [\log \pi_{\theta}(A_t | S_t) + \log p(S_{t+1} | S_t, A_t)].$$

Since the initial-state distribution $p(S_0)$ and transition dynamics $p(S_{t+1} | S_t, A_t)$ do not depend on θ , we have

$$\nabla_{\theta} \log p(\tau) = \sum_{t=0}^T \nabla_{\theta} \log \pi_{\theta}(A_t | S_t). \quad (17)$$

Substituting into (16):

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{\pi} \left[G(\tau) \sum_{t=0}^T \nabla_{\theta} \log \pi_{\theta}(A_t | S_t) \right]. \quad (18)$$

Expanding $G(\tau) = \sum_{k=0}^T \gamma^k R_{k+1}$ and exchanging the sums, we can remove rewards R_{k+1} for $k < t$ since they are independent of $\nabla_{\theta} \log \pi(A_t | S_t)$ given S_t (formally, $\mathbb{E}[\gamma^k R_{k+1} \nabla_{\theta} \log \pi(A_t | S_t) | S_t] = 0$ for $k < t$ by the baseline argument of Proposition 4.2, since past rewards are independent of the current action):

$$\begin{aligned} \nabla_{\theta} J(\theta) &= \mathbb{E}_{\pi} \left[\sum_{t=0}^T \left(\sum_{k=t}^T \gamma^k R_{k+1} \right) \nabla_{\theta} \log \pi(A_t | S_t) \right] \\ &= \mathbb{E}_{\pi} \left[\sum_{t=0}^T \gamma^t \left(\sum_{k=t}^T \gamma^{k-t} R_{k+1} \right) \nabla_{\theta} \log \pi(A_t | S_t) \right] \\ &= \mathbb{E}_{\pi} \left[\sum_{t=0}^T \gamma^t G_t \nabla_{\theta} \log \pi(A_t | S_t) \right] \\ &= \mathbb{E}_{\pi} \left[\sum_{t=0}^T \gamma^t q_{\pi}(S_t, A_t) \nabla_{\theta} \log \pi(A_t | S_t) \right], \end{aligned} \quad (19)$$

where the last equality holds because $\mathbb{E}_{\pi}[G_t | S_t, A_t] = q_{\pi}(S_t, A_t)$. $\square$

Remark 5.1 (Policy Gradients Do Not Require the Dynamics Model). Equation (14) involves only π_{θ} and q_{π} , not the transition probabilities $p(s' | s, a)$. This is surprising: shouldn't we need to know how the policy influences which states are visited? The score function trick absorbs this dependence into the expectation, so we only need *samples* from π_{θ} .

5.2 Practical Episodic Policy Gradient

Given a sampled episode, the gradient (14) can be estimated by pulling out the sum and applying per-timestep updates:

$$\Delta \theta_t = \gamma^t G_t \nabla_{\theta} \log \pi_{\theta}(A_t | S_t), \quad (20)$$

so that $\mathbb{E}_{\pi}[\sum_t \Delta \theta_t] = \nabla_{\theta} J(\theta)$.

Remark 5.2 (Dropping γ^t). In practice, the γ^t factor is often omitted, using instead $\Delta \theta_t = G_t \nabla_{\theta} \log \pi_{\theta}(A_t | S_t)$. This can be justified by pretending (at each step) that the episode could have started in S_t ; equivalently, it introduces a slight bias in the gradient.

5.3 Average-Reward Policy Gradient Theorem

Theorem 5.2 (Policy Gradient Theorem — Average Reward). *For any differentiable policy $\pi_{\theta}(a | s)$, the gradient of $J(\theta) = \mathbb{E}_{\pi}[R_{t+1}]$ (cf. Definition 3.2) is*

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{\pi}[q_{\pi_{\theta}}(S_t, A_t) \nabla_{\theta} \log \pi_{\theta}(A_t | S_t)], \quad (21)$$

where the expectation is over both states and actions, and the differential action-value function is

$$q_{\pi}(s, a) = \mathbb{E}_{\pi}[R_{t+1} - \rho + q_{\pi}(S_{t+1}, A_{t+1}) | S_t = s, A_t = a], \quad \rho = \mathbb{E}_{\pi}[R_{t+1}]. \quad (22)$$

Here ρ is the global average reward (not conditioned on state or action).

Theorem 5.3 (Policy Gradient Theorem — Average Reward, Alternative Form). *Equivalently, the average-reward policy gradient can be written as*

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{\pi} \left[R_{t+1} \sum_{n=0}^{\infty} \nabla_{\theta} \log \pi_{\theta}(A_{t-n} | S_{t-n}) \right], \quad (23)$$

where the expectation is over both states and actions.

6 Actor-Critic Methods

6.1 Reducing Variance with a Critic

By Proposition 4.2, we can subtract $v_{\pi}(S_t)$ from $q_{\pi}(S_t, A_t)$ in the policy gradient without changing its expectation:

$$\nabla_{\theta} J(\theta) = \mathbb{E} \left[\sum_{t=0}^T \gamma^t (q_{\pi}(S_t, A_t) - v_{\pi}(S_t)) \nabla_{\theta} \log \pi_{\theta}(A_t | S_t) \right]. \quad (24)$$

The quantity $A_{\pi}(s, a) := q_{\pi}(s, a) - v_{\pi}(s)$ is the **advantage function**.

In practice, we estimate the value function $v_{\mathbf{w}}(s) \approx v_{\pi}(s)$ (the *critic*) and use sampled returns or bootstrapped estimates for q_{π} :

- (i) **Monte Carlo**: $q_{\pi}(S_t, A_t) \approx G_t$.
- (ii) **Bootstrapping (TD)**: $q_{\pi}(S_t, A_t) \approx R_{t+1} + \gamma v_{\mathbf{w}}(S_{t+1})$, so the advantage estimate becomes the one-step TD error:

$$\delta_t = R_{t+1} + \gamma v_{\mathbf{w}}(S_{t+1}) - v_{\mathbf{w}}(S_t). \quad (25)$$

Bootstrapping further reduces variance at the cost of introducing bias from the value approximation (cf. `rl_lecture_5`, Theorem `thm:bias_var`).

6.2 The Critic

Definition 6.1 (Critic). A **critic** is a value function learned via *policy evaluation*: it answers the question “what is the value $v_{\pi_{\theta}}$ of the current policy π_{θ} ?” Any policy evaluation method can be used, for example: Monte-Carlo evaluation, TD(0) (cf. `rl_lecture_5`, Algorithm `alg:td0`), or n -step TD.

6.3 One-Step Actor-Critic Algorithm

Algorithm 6.1 (One-Step Actor-Critic). Maintain actor parameters θ and critic parameters $\mathbf{w}$. At each step:

1. Sample $A_t \sim \pi_{\theta}(\cdot | S_t)$.
2. Observe R_{t+1} and S_{t+1} .
3. Compute the TD error (advantage estimate):

$$\delta_t = R_{t+1} + \gamma v_{\mathbf{w}}(S_{t+1}) - v_{\mathbf{w}}(S_t).$$

4. **Critic update** (TD(0)):

$$\mathbf{w} \leftarrow \mathbf{w} + \beta \delta_t \nabla_{\mathbf{w}} v_{\mathbf{w}}(S_t).$$

5. **Actor update** (policy gradient, ignoring γ^t as in Remark 5.2):

$$\theta \leftarrow \theta + \alpha \delta_t \nabla_{\theta} \log \pi_{\theta}(A_t | S_t).$$

Remark 6.1 (Bad Policies Lead to Bad Data). In policy gradient methods, the data distribution depends on the policy being learned. A poor policy generates low-quality data, which can hinder further learning. This is fundamentally different from supervised learning, where the data is independent of the model. Many extensions and variants exist to address this instability.

6.4 Trust Region Methods

Definition 6.2 (KL-Regularised Policy Optimisation). To increase stability, we can **regularise** the policy update by penalising large changes. A popular approach limits the difference between successive policies using the Kullback–Leibler divergence:

$$\text{KL}(\pi_{\text{old}} \| \pi_{\theta}) = \mathbb{E} \left[\int \pi_{\text{old}}(a | S) \log \frac{\pi_{\theta}(a | S)}{\pi_{\text{old}}(a | S)} da \right], \quad (26)$$

where the expectation is over states. The optimisation objective becomes

$$\max_{\theta} J(\theta) - \eta \text{KL}(\pi_{\text{old}} \| \pi_{\theta}), \quad (27)$$

for some hyperparameter $\eta > 0$.

Remark 6.2 (Notable Trust Region Algorithms). This framework underlies several influential algorithms: TRPO (Schulman et al., 2015), PPO (Schulman et al., 2017), and MPO (Abdolmaleki et al., 2018).

7 Continuous Action Spaces

Pure value-based methods face difficulties in continuous action spaces: how to represent $q(s, a)$ when $a \in \mathbb{R}^d$, and how to compute $\max_a q(s, a)$? Policy-based methods handle continuous actions naturally, since we can parametrise the policy as any distribution over $\mathbb{R}^d$ and sample from it.

7.1 Gaussian Policy

Example 7.1 (Gaussian Policy). Let $\mu_{\theta}(s)$ be a parametric mean function and fix the variance at σ^2 (or parametrise it separately). The policy is

$$A_t \sim \mathcal{N}(\mu_{\theta}(S_t), \sigma^2). \quad (28)$$

The score function is

$$\nabla_{\theta} \log \pi_{\theta}(s, a) = \frac{A_t - \mu_{\theta}(S_t)}{\sigma^2} \nabla_{\theta} \mu_{\theta}(S_t). \quad (29)$$

Plugging into the policy gradient with baseline gives

$$\begin{aligned} \theta_{t+1} &= \theta_t + \beta (G_t - v(S_t)) \nabla_{\theta} \log \pi_{\theta}(A_t | S_t) \\ &= \theta_t + \beta (G_t - v(S_t)) \frac{A_t - \mu_{\theta}(S_t)}{\sigma^2} \nabla_{\theta} \mu_{\theta}(S_t). \end{aligned} \quad (30)$$

Intuition: if the return G_t exceeded the baseline $v(S_t)$, shift the mean $\mu_{\theta}(S_t)$ toward the action A_t that was taken.

7.2 Deterministic Policy Gradient (DPG)

Definition 7.1 (Deterministic Policy Gradient). When the critic generalises well, we can exploit it more directly. Given a learned $q_{\mathbf{w}}(s, a) \approx q_{\pi}(s, a)$ (e.g. via Sarsa) and a *deterministic* actor $A_t = \pi_{\theta}(S_t)$, improve the actor by gradient ascent on the value:

$$\Delta \theta \propto \frac{\partial Q_{\pi}(s, a)}{\partial \theta} = \frac{\partial Q_{\pi}(s, \pi_{\theta}(S_t))}{\partial \pi_{\theta}(S_t)} \frac{\partial \pi_{\theta}(S_t)}{\partial \theta}. \quad (31)$$

This is a form of policy iteration: evaluate the policy (critic), then improve it (actor) by ascending the value landscape.

Remark 7.1 (Historical Names). This idea has appeared under several names: “Action-dependent heuristic dynamic programming” (ADHDP; Werbos, 1990; Prokhorov & Wunsch, 1997), “Gradient ascent on the value” (van Hasselt & Wiering, 2007), and most recently, “Deterministic Policy Gradient” (DPG; Silver et al., 2014).

7.3 Continuous Actor-Critic Learning Automaton (CACLA)

Algorithm 7.1 (CACLA). An alternative continuous actor-critic that defines the error in *action space* rather than parameter space:

1. Get the current action proposal: $a_t = \text{Actor}_{\theta}(S_t)$.
2. Explore: $A_t \sim \mathcal{N}(a_t, \Sigma)$.
3. Compute TD error: $\delta_t = R_{t+1} + \gamma v_{\mathbf{w}}(S_{t+1}) - v_{\mathbf{w}}(S_t)$.
4. Update critic (e.g. TD(0)).
5. **If** $\delta_t > 0$: update actor toward the exploratory action:

$$\theta_{t+1} \leftarrow \theta_t + \beta (A_t - a_t) \nabla_{\theta} \text{Actor}_{\theta_t}(S_t).$$

6. **If** $\delta_t \leq 0$: do *not* update the actor.

Remark 7.2 (CACLA Properties). Two notable properties of CACLA: (i) the update magnitude does not depend on the value scale; (ii) the actor is never updated *away from* “bad” actions—it simply ignores them.

8 Summary

- (i) **Policy gradient:** directly optimise $J(\boldsymbol{\theta}) = \mathbb{E}[G_0]$ or $\mathbb{E}[R]$ by ascending $\nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta})$.
- (ii) **Score function trick** (Lemma 4.1): $\nabla_{\boldsymbol{\theta}} \mathbb{E}_{\pi}[f] = \mathbb{E}_{\pi}[f \nabla_{\boldsymbol{\theta}} \log \pi]$ enables gradient estimation from samples.
- (iii) **Policy gradient theorem** (Theorems 5.1 and 5.2): extends the bandit result to multi-step MDPs; the gradient does *not* require the transition model.
- (iv) **Baselines** (Proposition 4.2): subtracting a state-dependent baseline (e.g. $v_{\pi}(s)$) reduces variance without bias.
- (v) **Actor-critic** (Algorithm 6.1): combining a policy (actor) with a value function (critic) enables lower-variance, online updates.
- (vi) **Continuous actions:** policy gradients naturally handle continuous action spaces via parametric distributions (e.g. Gaussian).
- (vii) **DPG** (Definition 7.1): for deterministic policies, ascend the critic's value directly, bypassing the score function.
- (viii) **Trust regions** (Definition 6.2): regularise with $\text{KL}(\pi_{\text{old}} \parallel \pi_{\boldsymbol{\theta}})$ for stability.